Projeto de Tecnologias de
Informação e Redes
2025/2026

Enunciado - Componente Redes

v2.0

1 Introdução

Este projeto tem como objetivo aplicar, de forma integrada, os conceitos estudados nas unidades curriculares da área de redes, sistemas distribuídos e desenvolvimento de software. Ao longo do semestre, os estudantes deverão desenhar e implementar um sistema distribuído completo, aplicando boas práticas de engenharia de software e de arquitetura de sistemas.

O trabalho deverá demonstrar a capacidade de:

- analisar um problema e propor uma arquitetura distribuída adequada.
- definir uma interface de comunicação entre serviços, nomeadamente através de uma API REST.
- identificar e justificar requisitos não funcionais, como disponibilidade, escalabilidade e segurança.
- escolher, implementar e configurar os diferentes componentes do sistema.
- validar o funcionamento do sistema através de testes.

O projeto está organizado em **quatro** fases principais, que acompanham a evolução natural do desenvolvimento do sistema ao longo do semestre:

1. **Conceção (Design)**
   Nesta fase é definida a arquitetura geral do sistema distribuído. Inclui a identificação dos requisitos não funcionais e o desenho das arquiteturas que irão suportar o sistema.
2. **Implementação da Arquitetura Distribuída**
   Implementação da infraestrutura distribuída proposta, incluindo a criação e configuração das instâncias dos balanceadores de carga, servidores aplicacionais e bases de dados, bem como das redes necessárias à comunicação entre os diferentes componentes.
3. **Segurança**
   Configuração dos mecanismos de segurança do sistema, utilização de canais seguros, configuração de firewall.
4. **Automação e Testes**
   Implementação de mecanismos de automação para construção e lançamento da aplicação, bem como testes de carga e testes de vulnerabilidades para validar o funcionamento e a robustez do sistema.

Ao longo do semestre serão realizadas **reuniões de avaliação intermédias**, nas quais cada grupo deverá apresentar as decisões de design tomadas (1ª reunião) e demonstrar o progresso do trabalho (2ª,3ª e 4ª reuniões).

O objetivo final é que cada grupo desenvolva uma solução que integre princípios fundamentais de sistemas distribuídos, demonstrando capacidade de conceção, implementação e operação de um sistema realista em ambiente de produção.

---

2 Conceção (Design)

Nesta fase, cada grupo deverá definir a conceção do sistema distribuído que irá desenvolver ao longo do projeto. O objetivo é especificar a arquitetura geral do sistema, a interface de comunicação entre componentes e os requisitos não funcionais que a infraestrutura deverá satisfazer.
Na fase de Design, o objetivo não deve ser implementar nada. Deve ser pensar e justificar a arquitetura antes de a construir. O foco deve estar na definição, justificação e documentação das decisões de arquitetura. Esta fase termina com a elaboração de um documento: Relatório de Design. Este deverá incluir quatro secções, nomeadamente:

1. Visão geral do sistema
2. Requisitos Não Funcionais (NFR)
3. Arquitetura do Sistema
4. Arquitetura de Segurança

2.1 Visão geral do sistema

Na primeira seclção do relatório deverá ser apresentada uma breve descrição do sistema a desenvolver, incluindo (1 página):

1. objetivo do sistema
2. contexto de utilização
3. principais componentes do sistema
4. breve descrição das interações entre os componentes

Esta secção serve para contextualizar a solução proposta e explicar o papel de cada componente no sistema distribuído.

2.2 Requisitos Não Funcionais (NFR)

Na segunda secção do relatório deverão ser apresentados os Requisitos Não Funcionais (NFR) que descrevem propriedades de qualidade do sistema distribuído, como disponibilidade, desempenho, segurança ou escalabilidade. Ao contrário dos requisitos funcionais, que definem o que o sistema faz, os requisitos não funcionais definem como o sistema deve comportar-se e operar. Nesta disciplina, a definição e análise dos requisitos não funcionais constitui uma parte fundamental da fase de Design, uma vez que estes requisitos influenciam diretamente o desenho da arquitetura distribuída. Cada grupo deverá identificar e especificar um conjunto de **sete** requisitos não funcionais, correspondentes aos temas indicados abaixo. Para cada requisito deverão:

- descrever claramente o requisito
- indicar uma métrica objetiva que permita avaliar o seu cumprimento
- explicar que mecanismos arquiteturais ou técnicos serão utilizados para satisfazer esse requisito

O objetivo desta secção é demonstrar que o grupo consegue relacionar requisitos de qualidade com decisões de arquitetura. No relatório, para cada requisito deverá ser utilizada a seguinte estrutura:

Code/Number	Código ou número identificativo do requisito.
Topic	Um dos sete temas
Requirement	Descrição clara do requisito.

Metric	Forma objetiva de verificar se o requisito é cumprido.
Mechanism	Mecanismos arquiteturais ou técnicos utilizados para satisfazer o requisito.

2.2.1 Temas dos Requisitos Não Funcionais

Os grupos deverão definir um requisito para cada um dos seguintes temas.

NFR 1 – Disponibilidade
Perguntas orientadoras:
- O sistema continua a responder se um servidor falhar?
- Qual o tempo máximo aceitável de indisponibilidade?
- Existem componentes que constituem um ponto único de falha?

NFR 2 – Escalabilidade
Perguntas orientadoras:
- O que acontece se o número de utilizadores aumentar significativamente?
- É possível adicionar novos servidores?
- Como é distribuída a carga entre instâncias?

NFR 3 – Desempenho
Perguntas orientadoras:
- Qual o tempo de resposta esperado para pedidos à API?
- Quantos pedidos por segundo o sistema deverá suportar?
- Que componentes podem tornar-se gargalos?

NFR 4 – Tolerância a Faltas
Perguntas orientadoras:
- Como é detetada a falha de um servidor?
- O sistema consegue continuar a funcionar após uma falha?
- Existe algum mecanismo de substituição automática de instâncias?

NFR 5 – Segurança
Perguntas orientadoras:
- Como são autenticados os utilizadores ou clientes da API?
- O tráfego entre clientes e servidores é protegido?
- Que mecanismos existem para impedir acessos indevidos?

NFR 6 – Operação e Manutenção do Sistema
Perguntas orientadoras:
- Como são feitas novas versões do sistema?
- É possível reconstruir e lançar o sistema automaticamente?
- Como é gerido o código e as contribuições da equipa?

NFR 7 Isolamento e Segmentação de Rede
Perguntas orientadoras:

- Existem redes separadas para clientes e serviços internos?
- Que componentes estão acessíveis a partir da Internet?
- Como é controlado o acesso entre diferentes redes?

2.3 Arquitetura do Sistema

Após a definição dos requisitos não funcionais, cada grupo deverá apresentar o desenho da arquitetura distribuída do sistema que será implementado ao longo do projeto. Uma versão simplificada da arquitetura sugerida é apresentada na figura abaixo.

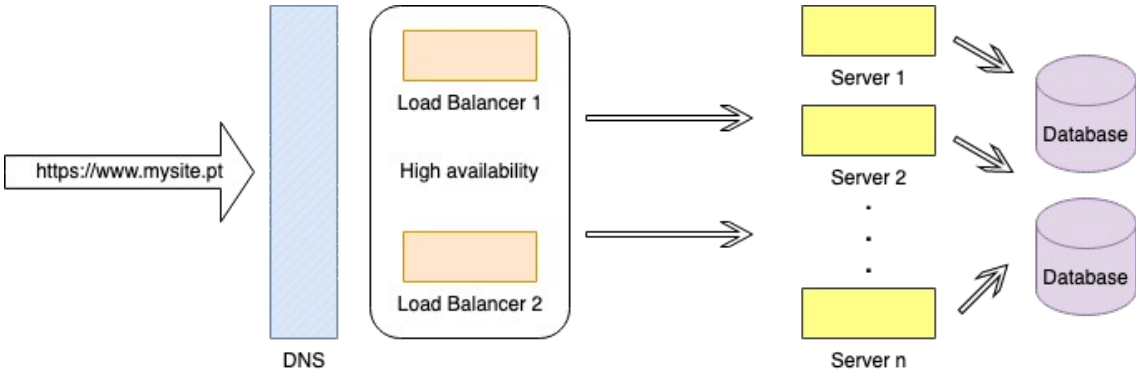

Fig.1 Diagrama da Arquitetura do Sistema sugerida

A partir desta versão, os grupos deverão desenhar uma arquitetura completa do sistema, contendo os detalhes necessários para a sua implementação. ,Na terceira secção do seu Relatório de Design deverão ser apresentados quatro subsecções, nomeadamente:

1. Diagrama de Sistema (componentes e redes)
2. Descrição da estratégia de redundância e tolerância a falhas.
3. Descrição da estratégia de escalabilidade
4. Tabela de comunicações entre componentes (portos e protocolos)

A arquitetura deverá incluir três tipos principais de componentes:

I. Balanceador de carga
II. Servidor aplicacional
III. Base de dados

Todos os componentes deverão ser executados em máquinas virtuais distintas e comunicarem através da rede definida na arquitetura.

2.3.1 Balanceadores de carga

Os balanceadores de carga são responsáveis por receber os pedidos dos clientes e distribuí-los pelos diferentes servidores aplicacionais. Para evitar que este componente constitua um ponto único de falha, deverão existir pelo menos duas instâncias de balanceador de carga configuradas em modo de alta disponibilidade (High Availability).

Existem duas configurações possíveis:

- Ativo-passivo, em que um balanceador executa o serviço e o segundo assume o funcionamento em caso de falha do primeiro
- Ativo-ativo, em que ambos os balanceadores prestam serviço simultaneamente

Neste projeto deverá ser utilizada a configuração ativo-passivo.

2.3.2 Servidores aplicacionais

Os servidores aplicacionais executam a lógica da aplicação e respondem aos pedidos encaminhados pelos balanceadores de carga. O sistema deverá possuir pelo menos duas instâncias de servidor, permitindo distribuir os pedidos e aumentar a disponibilidade do sistema. A arquitetura deverá permitir escalabilidade horizontal, possibilitando a adição de novas instâncias de servidor.

A escalabilidade poderá ser realizada de duas formas:

- Escalabilidade passiva, em que novas instâncias são adicionadas manualmente e registadas no balanceador de carga
- Escalabilidade ativa, em que o sistema monitoriza a carga e adiciona ou remove instâncias automaticamente

Para efeitos deste projeto deverá implementar escalabilidade passiva.

2.3.3 Base de dados

A base de dados deverá ser implementada com duas instâncias, configuradas em modo de redundância. Uma das instâncias deverá funcionar como base de dados principal, enquanto a segunda deverá funcionar como instância de reserva, assumindo o funcionamento em caso de falha da primeira.

2.3.4 Arquitetura de Rede

A arquitetura do sistema deverá incluir **sub-redes distintas**, com o objetivo de separar diferentes tipos de tráfego e aumentar a segurança do sistema. Deverão existir pelo menos duas redes:

- **rede externa**, utilizada para receber pedidos dos clientes
- **rede interna**, utilizada para comunicação entre os componentes do sistema

Os balanceadores de carga deverão estar ligados a ambas as redes, enquanto os servidores aplicacionais e a base de dados deverão estar acessíveis apenas através da rede interna.

2.4 Arquitetura de Segurança

O sistema deverá incluir mecanismos que garantam a proteção dos serviços e dos dados, evitando acessos não autorizados e exposição desnecessária de componentes internos. No relatório de Design os grupos deverão descrever os mecanismos de segurança previstos para o sistema.

2.4.1 Exposição de serviços

Os grupos deverão identificar quais os componentes do sistema que estão acessíveis a partir da Internet. Regra geral esperada:

- apenas o **balanceador de carga** deverá estar acessível a partir da Internet
- os **servidores aplicacionais** deverão estar acessíveis apenas através da rede interna
- a **base de dados** não deverá estar diretamente acessível a partir da Internet

---

2.4.2 Controlo de acesso à rede

Os grupos deverão definir regras de controlo de acesso entre os diferentes componentes do sistema. Estas regras podem ser implementadas através de:

- regras de firewall
- security groups da plataforma cloud
- iptables nas máquinas virtuais

O relatório deverá indicar que componentes podem comunicar entre si e em que portas ou protocolos.

2.4.3 Comunicação segura

O sistema deverá utilizar canais seguros para comunicação com clientes externos.
Os grupos deverão indicar:

- utilização de HTTPS

- utilização de certificados TLS
- associação do sistema a um nome de domínio

2.4.4 Acesso administrativo

Os grupos deverão indicar como será realizado o acesso administrativo às máquinas virtuais, nomeadamente:

- utilização de **SSH**
- utilização de **chaves SSH**
- eventuais restrições de acesso por endereço IP ou rede

---

3 Automação e Testes

Na fase final do projeto, os grupos deverão implementar mecanismos que permitam automatizar a construção e o lançamento do sistema, bem como validar o seu funcionamento através de diferentes tipos de testes. O objetivo desta fase é garantir que o sistema pode ser instalado, atualizado e verificado de forma reprodutível, reduzindo erros manuais e aumentando a fiabilidade da infraestrutura.

3.1 Automação do deployment

Os grupos deverão desenvolver mecanismos que permitam instalar, configurar e lançar automaticamente os diferentes componentes do sistema distribuído. O objetivo desta fase é garantir que a infraestrutura pode ser recriada de forma consistente, reduzindo a necessidade de configuração manual.

A automação deverá incluir:

- instalação e configuração dos serviços necessários em cada máquina virtual
- configuração dos balanceadores de carga
- configuração dos servidores aplicacionais
- configuração das bases de dados
- configuração das regras de rede e firewall

A automação deverá ser realizada através de scripts de instalação e configuração. Os grupos deverão disponibilizar vários scripts, listados abaixo.

3.1.1 Servidores aplicacionais

- `create_app_server.sh`
  Cria uma nova instância de servidor aplicacional, instala e configura a aplicação, inicia o serviço e adiciona o servidor ao balanceador de carga.
- `kill_app_server.sh`
  Termina uma instância específica de servidor aplicacional, identificada por um parâmetro (por exemplo, o nome ou identificador da instância), permitindo demonstrar o comportamento do sistema quando um servidor deixa de responder.

3.1.2 Base de dados

- `create_db_primary.sh`
  Cria e inicia a instância da base de dados principal.
- `create_db_backup.sh`
  Cria e inicia a instância da base de dados de reserva.

- `kill_db_primary.sh`
  Termina a instância da base de dados principal, permitindo demonstrar o comportamento do sistema quando a DB deixa de responder.
- `promote_db_backup.sh`
  Promove a instância de base de dados de reserva a base de dados principal após a falha da instância principal.

3.1.3 Balanceadores de carga

- `create_lb.sh`
  Cria e inicia um balanceador de carga principal (LB1). Associa a este o endereço IP público estático reservado do sistema,
- `create_lb_backup.sh`
  Cria e inicia o balanceador de carga de reserva (LB2). Configurado (p. ex com HAProxy) para encaminhar pedidos para os servidores aplicacionais.
- `kill_lb_primary.sh`
  Termina o processo do balanceador de carga na instância LB1 (por exemplo, o processo HAProxy), permitindo demonstrar o impacto da falha do serviço sem desligar a máquina virtual.
- `promote_lb_backup.sh`
  Promove o balanceador de carga de reserva (LB2) a balanceador ativo, reatribuindo o endereço IP público estático reservado do sistema para a instância LB2.

3.2 Demonstração final

Durante a demonstração final os grupos deverão ser capazes de:

- lançar novas instâncias de servidor aplicacional e demonstrar a integração no balanceador de carga
- criar a base de dados principal e a base de dados de reserva
- terminar a base de dados principal e observar o impacto no sistema
- iniciar LB1 e LB2 em modo ativo-passivo
- terminar LB1 e demonstrar o comportamento do sistema perante essa falha

---

3.3 Testes de carga

Os grupos deverão realizar **testes de carga**, com o objetivo de avaliar o comportamento do sistema quando sujeito a múltiplos pedidos simultâneos.
Estes testes deverão permitir verificar:

- o tempo de resposta do sistema
- a capacidade do sistema suportar múltiplos clientes
- o comportamento do balanceamento de carga

Ferramentas recomendadas:

- **Locust**
- outras ferramentas equivalentes

3.4 Testes de vulnerabilidade

Os grupos deverão realizar testes básicos de segurança com o objetivo de identificar possíveis vulnerabilidades no sistema.

Estes testes poderão incluir:

- análise de portas abertas
- identificação de serviços expostos

Ferramentas recomendadas:

- nmap
- OWASP ZAP
- outras ferramentas equivalentes

---

4 Infraestrutura

É aconselhada a utilização de máquinas virtuais num fornecedor de serviços em nuvem. Os grupos deverão optar por uma das seguintes plataformas:

- **Google Cloud Platform (GCP)**
- **Amazon Web Services (AWS)**
- **Microsoft Azure**

Desta forma o sistema poderá ser acedido tanto pelos elementos do grupo como pelos docentes.

---

5 Restrições de implementação

Não deverão ser utilizados serviços geridos que forneçam automaticamente funcionalidades de balanceamento de carga, escalabilidade ou tolerância a falhas.

Exemplos de serviços que não devem ser utilizados:

- Google App Engine
- Kubernetes
- plataformas PaaS equivalentes

Espera-se que cada grupo implemente estes mecanismos diretamente sobre máquinas virtuais, instalando e configurando as aplicações necessárias. O uso de containers poderá ser utilizado para facilitar o desenvolvimento e o lançamento em produção.

---

6 Avaliação (Continua)

O desenvolvimento do projeto será avaliado ao longo do percurso em reuniões de avaliação periódica, conforme abaixo especificado. Cada grupo deverá entregar a documentação abaixo descrita ao longo da execução do projeto.

			Pesos
Design (25%)	1	Relatório Design: Visão Geral	3,0%
	2	Relatório Design: NFRs	7,0%
		(cada)	(1,0%)
	3	Relatório Design: Arquitetura distribuída	11,0%
	3.1	Diagrama de Sistema (componentes e redes)	5,0%
	3.2	Descrição da estratégia de redundância e tolerância a falhas.	2,0%
	3.3	Descrição da estratégia de escalabilidade	2,0%
	3.4	Tabela de comunicações entre componentes (portos e protocolos)	2,0%
	4	Relatório Design: Arquitetura Segurança	4,0%
	4.1	Acesso administrativo	1,0%

	4.2	Comunicação segura	1,0%
	4.3	Controlo de acesso à rede	1,0%
	4.4	Exposição de serviços	1,0%
Arq. distribuída (25%)	5	Implementação da arquitetura distribuída	25,0%
	5.1	Implementação dos LBs	7,5%
	5.2	Implementação dos Servidores Aplicacionais	10,0%
	5.3	Implementação das BDs	7,5%
Arq. Segurança (20%)	6	Canais seguros, DNS e configuração de firewall	10,0%
	6.1	Uso de TLS com certificado assinado por uma Autoridade Certificadora (AC)	5%
	6.2	Nome de domínio registado e associado ao IP estático	5%
	6.3	Configurações de firewall	10%
Automação e Testes (30%)	7	Automação e scripts para construção e lançamento da aplicação	20,00%
	7.1	Scripts de Servidor Aplicacional	4,0%
	7.2	Scripts de LBs	8,0%
	7.3	Scripts de DBs	8,0%
	8	Testes de carga	5,00%
	9	Testes de vulnerabilidades	5,00%

7 Guiões e tutoriais

Um conjunto de guiões e tutoriais que podem usar como base para o desenvolvimento do projeto é apresentado abaixo:

Orientações gerais	https://12factor.net/
GitHub Hello World	https://guides.github.com/activities/hello-world/
GitHub Actions Hello World	https://docs.github.com/pt/actions/writing-workflows/quickstart
OpenAPI	https://swagger.io/specification/
OpenAPI Tutorial	https://support.smartbear.com/swaggerhub/docs/tutorials/openapi-3-tutorial.html
Cliente para testes da API	https://www.postman.com
OpenAPI editor especificação	https://editor.swagger.io/
OpenAPI gerador da especificação (alternativa ao editor)	https://www.apimatic.io/
Load Balancer - HAProxy	http://cbonte.github.io/haproxy-dconv/2.5/intro.html e Guião "Balanceadores de Carga / HAProxy" da disciplina de PTR
Load Balancer - Nginx	https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/
Nginx - Docker container	https://hub.docker.com/_/nginx
Monitorização	https://www.netdata.cloud
Segurança	https://owasp.org/
Top 10 OWASP	https://owasp.org/www-project-top-ten/
Autenticação	https://auth0.com/
Reserva de nome de domínio gratuito	Devem ser identificados os serviços que oferecem nomes gratuitamente no arranque do projeto de PTR como domínios.pt
Certificados SSL	https://letsencrypt.org/ , certbot
Testes unitários	Python: https://docs.python.org/3/library/unittest.html, PHP: https://phpunit.de/
Testes de carga	https://locust.io/
Testes de vulnerabilidade	https://nmap.org/, https://www.zaproxy.org/, https://w3af.org/, https://sqlmap.org/